Flask Day-2

Class Presentation

Environment Setup & First Flask Application

Setup + First App | venv + pip | debug mode

Trainer

Madhu Parvathaneni, CTO - Codegnan
madhu@codegnan.com - 8121289993

codegnan®

01 **Python** Verify installation & core environment

02 **VS Code** Configure terminal & extensions

03 **Virtual Env** Isolate project using venv & pip

04 **Flask App** Write initial web application server

05 **Browser** Test local routing & live outputs

What students must master today

The class should end with every student running Flask locally.

01

Install & verify Python

python --version and pip --version

02

Use VS Code properly

Open folder, terminal, extensions

03

Create virtual environment

Project isolation using venv

04

Install Flask

pip install flask in the correct env

05

Create first app.py

Flask object, route, function, return

06

Run, debug & auto reload

Use debug=True and observe changes

PART 1

Setup the development environment

Install the tools and verify that the system is ready before writing Flask code.

01 **Python** Verify installation & PATH

02 **VS Code** Configure terminal & extensions

03 **Virtual Env** Isolate project using venv

04 **Flask App** Install and run first server

Environment setup: the big picture

Before Flask, students need a working local development system.

Step 1: Python installation & verification

Students must verify Python and pip before creating a Flask project.

1 What to install

Use Python 3.10+ or 3.12+. During installation, enable **Add Python to PATH**.

```
python --version
py --version
pip --version
```

2 Expected result

Terminal should show Python version and pip version without command-not-found errors.

```
Python 3.12.2
pip 24.0 from ... (python 3.12)
```

Windows troubleshooting

Problem	Likely Reason	Fix
python is not recognized	PATH not added	**Reinstall Python or add PATH**
pip is not recognized	pip not configured	**Use `py -m pip --version`**
Wrong Python version	Multiple versions	**Use `py -3.12` or check PATH**

Step 2: VS Code setup

VS Code is where students will write, run and debug Flask applications.

01 Open folder

Always open the project folder, not a single file. This keeps terminal and paths correct.

02 Terminal

Use Terminal > New Terminal. Commands should run from the project directory.

03 Extensions

Python, Pylance, Auto Rename Tag, Thunder Client or Postman.

04 Interpreter

Select the virtual environment interpreter after creating venv.

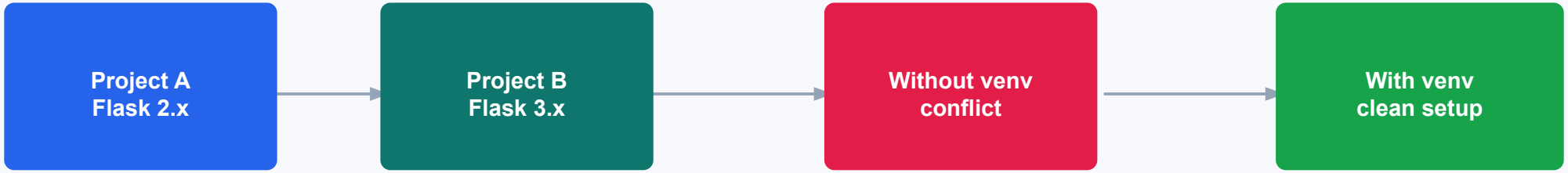
```
File Open Folder
Terminal New Terminal
Ctrl + Shift + P Python: Select Interpreter
```

Class activity

Ask every student to create one folder named `flask_day2` and open it inside VS Code. Then run `pwd` / `cd` command to confirm location.

Virtual environment: why do we need it?

A virtual environment keeps project packages separate and controlled.



📦 Simple explanation

venv is a separate box for project packages. Each project can have its own Flask and libraries.

🏫 Real-world analogy

Like separate classrooms for different batches; materials do not get mixed.

⭐ Exam/industry point

Professional projects always use isolated environments and requirements.txt.

Step 3: Create and activate virtual environment

Run these commands from inside the project folder.

```
# Create virtual environment
python -m venv venv

# Windows activation
venv\Scripts\activate

# macOS / Linux activation
source venv/bin/activate
```

How to know it worked?

```
(venv) C:\Users\Student\flask_day2>
```

⚠️ **Important**

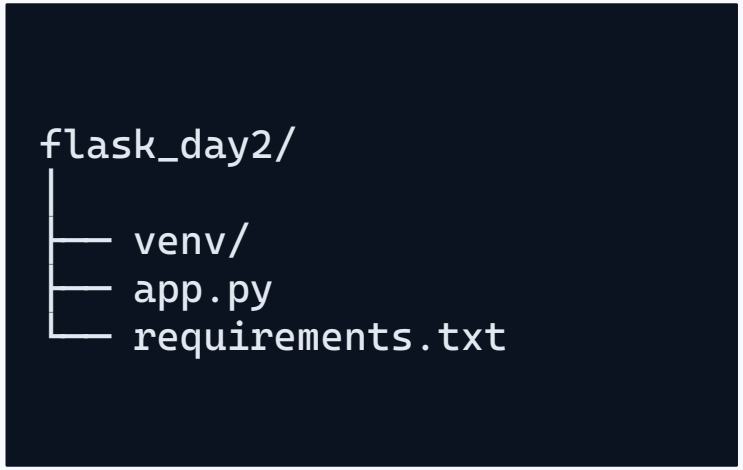
The terminal prompt should show (venv). If not visible, Flask may install in the wrong place.

✅ **Trainer check**

Ask students to raise hand only after they see (venv) in terminal.

Step 4: Project directory structure

A clean structure avoids confusion as the project grows.



📄 app.py

Main Flask application file. It creates app object, routes and response functions.

📦 venv/

Private environment that stores project packages.

🗒️ requirements.txt

List of packages required to run the project on another system.

🧩 templates/ later

HTML files will be placed here from upcoming classes.

Avoid spaces in folder names. Use simple names like flask_day2, first_flask_app, student_notes.

Step 5: Install Flask inside venv

After activation, install Flask using pip and verify it.

```
pip install flask

python -c "import flask; print(flask.__version__)"

pip freeze
```

📥 What pip does

pip downloads and installs Python packages into the currently active environment.

📋 What pip freeze does

Shows packages currently installed. This list can be saved into requirements.txt.

```
pip freeze > requirements.txt
```

Micro question: If Flask is installed but Python cannot import it, what should we check first? Answer: active environment / interpreter.

PART 2

Build the first Flask application

Now students move from setup to a running web app.

First Flask application: app.py

This is the smallest meaningful Flask application students should write today.

```
from flask import Flask

app = Flask(__name__)

@app.route("/")
def home():
    return "Hello from Flask Day 2!"

if __name__ == "__main__":
    app.run(debug=True)
```

✍ Student task

Type the code manually. Do not copy-paste. Typing builds syntax familiarity.

✅ Expected output

Browser displays: Hello from Flask Day 2!

⚠ Most common mistake

Missing colon after function or wrong indentation below def home().

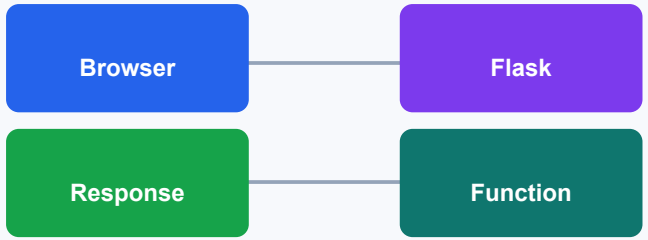
Understand every line

Students should be able to explain the app, not only run it.

Line	Meaning
from flask import Flask	Import Flask class from flask package
app = Flask(__name__)	Create Flask application object
@app.route("/")	Map root URL to the next function
def home():	Function that handles the request
return "Hello"	Response sent back to browser
app.run(debug=True)	Start local server with debug mode

🧠 Key mental model

URL comes to Flask. Flask checks routes. Matched function returns response.



Step 6: Run the Flask server

Run the application from terminal and open the local URL.

```
python app.py
```

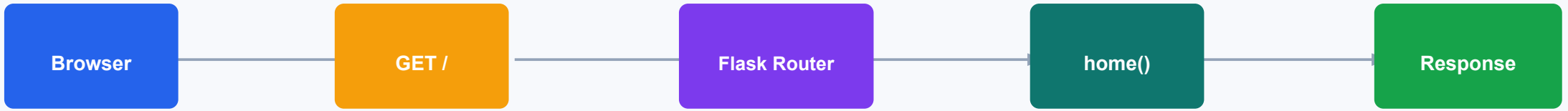
Expected terminal output

```
* Serving Flask app "app"
* Debug mode: on
* Running on http://127.0.0.1:5000
```

Teach this clearly: 127.0.0.1 means this computer itself. It is not a public website.

What happens when browser opens 127.0.0.1:5000?

Connect Day-1 request-response theory with today's practical output.



Browser

Sends a request to the local Flask server.

Router

Checks @app.route("/") and decides which function to run.

Function

home() returns a string response to the browser.

Debug mode: useful but only for development

Debug mode helps learning because errors are visible and code reloads automatically.

```
app.run(debug=True)
```

✅ Benefit 1

Shows detailed error pages when code fails.

✅ Benefit 2

Restarts server automatically when files are saved.

⚠️ Warning

Never use debug=True in production deployment.

Trainer demo: Change the return text, save the file, refresh browser, and show that Flask auto reloads.

Common Day-2 errors and quick fixes

This slide is for live troubleshooting during class.

Error	Why it happens	Fix
ModuleNotFoundError: flask	Flask not installed in active env	Activate venv, then pip install flask
Address already in use	Port 5000 already running	Stop old server or use another port
IndentationError	Function body not indented	Indent return line with 4 spaces
SyntaxError	Missing colon or quote	Read exact line number and fix syntax
Page not loading	Server not running / wrong URL	Run python app.py, open 127.0.0.1:5000

Class rule: Read the error message before asking for help. The error usually tells the line and the problem.

Hands-on lab: build three routes

Students must complete this before the class ends.

```
@app.route("/")
def home():
    return "Home Page"

@app.route("/about")
def about():
    return "About Page"

@app.route("/contact")
def contact():
    return "Contact Page"
```

Acceptance criteria

- Project folder opened in VS Code
- venv created and activated
- Flask installed inside venv
- app.py runs without error
- /, /about and /contact return correct text
- Student can explain @app.route()

⭐ Stretch task

Add /profile route and return your name, batch and today's topic.

Quick class quiz

Ask these before closing the session.

1	**Why do we create a virtual environment?**	To isolate project packages
2	**Which file starts our Flask app today?**	app.py
3	**What does @app.route("/") do?**	Maps URL / to a function
4	**What does debug=True help with?**	Error visibility and auto reload
5	**What is 127.0.0.1?**	Local computer / localhost
6	**What should you check if Flask import fails?**	venv and selected interpreter

Teaching close: Students should understand commands as a workflow, not as random terminal text.

Day-2 Assignment

Students should submit GitHub evidence or screenshots after completing this.

1 Task 1

Create a Flask app with five routes: /, /about, /courses, /contact, /profile.

2 Task 2

Each route should return a different meaningful sentence.

3 Task 3

Add comments explaining Flask object, route decorator and app.run().

4 Task 4

Create requirements.txt using pip freeze > requirements.txt.

5 Task 5

Capture output screenshots for all five routes.

6 Task 6

Push the project to your own GitHub repository.

Submission folder name: PFS-HYD-054_Flask_Day2_<YourName>

Ready for Day-3 Routing

Today we successfully set up the development environment, initialized our workspace, and executed the first running Flask application.

Tomorrow, we scale this architectural foundation into proper dynamic routing and request handling.

Day 3 Focus

Transitioning from single-file execution to scalable Flask structures, dynamic route variables, custom URL converters, and view templates.

Milestones Checked

- ✓ **Python environment verified**
- ✓ **VS Code installation ready**
- ✓ **Virtual environment (venv) created**
- ✓ **Flask installed within environment**
- ✓ **app.py logic completed**
- ✓ **Local development server running**